
Energy Efficient Patent Classification on Text-attributed Graph

Homaira Huda Shomee

Department of Computer Science
University of Illinois Chicago
hshome2@uic.edu

Ataher Sams

Department of Electrical and Computer Engineering
University of Illinois Chicago
asams3@uic.edu

1 Introduction

Patent classification is a fundamental task in intellectual property management, playing a crucial role in organizing patent documents and supporting services such as prior-art search, portfolio analysis, and technology trend assessment. In the patenting process, examiners traditionally assign each patent to one or multiple categories based on its technical domain. Effective automation of patent classification is therefore urgently needed to streamline patent review and improve retrieval and analytics in large-scale patent databases. An accurate classification system enables easier navigation of millions of patents, helps identify relevant prior art, and aids innovators and attorneys in managing IP assets efficiently. An example USPTO patent is shown in Figure 1. Early approaches to automatic patent classification [7, 1] treated it as a text categorization problem, leveraging the fact that patents contain rich textual fields (titles, abstracts, claims, etc.). Traditional methods relied on handcrafted features or bag-of-words representations of patent text, coupled with classifiers like support vector machines (SVMs) or rule-based systems. With the rise of machine learning, more sophisticated models were applied: for instance, DeepPatent [10] employed a convolutional neural network (CNN) on patent text embeddings to improve classification accuracy, and PatentBERT [9] fine-tuned a BERT transformer model for multi-label patent categorization. These deep learning models significantly outperformed earlier methods by capturing semantic nuances in patent language. However, they largely focused on textual information alone, ignoring other patent data modalities. Patents also come with structured metadata (e.g. inventors, assignees, technological categories) and citation links to other patents; such information was often used only in limited ways, if at all.

Recent research has moved toward integrating textual and structural data to overcome these limitations. A patent database can be represented as a graph: each patent is a node and edges can denote relationships like citations, shared inventors, or common technological fields. Graph Neural Networks (GNNs) provide a framework to exploit such structure for classification. Initial graph-based patent classifiers augmented each patent node with a text-derived feature vector and applied standard GNN models (e.g. GCNs [8] and GraphSAGE [5]) to propagate information through the citation network. For example, one study proposed an attention-based Graph Convolutional Network over a patent citation graph (with text features on nodes) to perform classification. The intuition is that combining semantic content of patents with their citation context can improve prediction: a patent’s neighbors may hint at its field, and network propagation can smooth out classification decisions among related patents. Beyond citations, patents are linked via various metadata, forming a multi-view graph. In this direction, Patent2Vec [3] introduced a multi-view representation learning paradigm for patent classification. A limitation of Patent2Vec, however, is the complexity of constructing and processing multiple large graphs; it requires comprehensive patent metadata and careful alignment of information across views. Moreover, while it captures “words” as one view, it does not employ advanced language models for deep semantic understanding of the text, relying more on keyword-level connections. This leaves room for approaches that more directly harness textual semantics alongside graph structure.

An emerging frontier in text-attributed-graph is the integration of LLMs and Graph Neural GNNs, leveraging the strengths of both architectures—LLMs for their contextual understanding of text and

US 20220092440A1

(19) **United States**
(12) **Patent Application Publication**
Friedrich et al.

(10) **Pub. No.: US 2022/0092440 A1**
(43) **Pub. Date: Mar. 24, 2022**

Title (54) **DEVICE AND METHOD FOR DETERMINING A KNOWLEDGE GRAPH**

Applicant (71) **Applicant: Robert Bosch GmbH, Stuttgart (DE)**

Inventor (72) **Inventors: Annemarie Friedrich, Weil Der Stadt (DE); Subhash Chandra Pujari, Renningen (DE)**

Application Number (21) **Appl. No.: 17/400,631**

(22) **Filed: Aug. 12, 2021**

(30) **Foreign Application Priority Data**
Sep. 21, 2020 (EP) 20197168.6

Publication Classification

(51) **Int. Cl.**
G06N 5/02 (2006.01)
G06N 3/04 (2006.01)
G06K 9/62 (2006.01)

(52) **U.S. CL**
CPC G06N 5/02 (2013.01); G06N 3/04 (2013.01); G06K 9/6296 (2013.01); G06K 9/6282 (2013.01); G06K 9/6257 (2013.01)

Abstract (57)
A device and a computer-implemented method of determining a knowledge graph. The method includes determining an embedding for a sequence of tokens of an instance, determining a first classification for the contextual embedding at a first classifier, determining if the first classification meets a first condition, adding to the knowledge graph a first link between a first node of the knowledge graph representing the instance and a node of the knowledge graph representing the first classification when the first classification meets the first condition and not adding the first link otherwise.

Figure 1: The bibliographic information for an example USPTO patent with the publication number “US 2022/0092440 A1” [4]

GNNs for their ability to model relational and structural information in graph-based data. A general approach in recent research involves extracting embeddings from various pre-trained language models and incorporating them into different GNN architectures. This hybrid framework aims to improve both model performance and efficiency. GLEM [13] employs an Expectation-Maximization (EM) framework that alternates between updating the language model in the E-step and the GNN in the M-step. This decoupled training strategy allows the two modules to be trained independently while still enabling mutual interaction and enhancement. Another example is TAPE [6], which enriches textual node features using LLM-generated explanations to improve the performance of GNNs on downstream tasks. Specifically, it incorporates these explanations as additional node features in node classification problems. However, despite the promising potential of such LLM-GNN integrations, they have not yet been widely applied to patent datasets. This represents a key research opportunity to explore how combining contextual language understanding with relational graph structures could benefit complex task like patent classification.

2 Problem Statement

We consider the patent data as a Text-Attributed Graph (TAG), an information network where each node is associated with a textual description. In this TAG, each node corresponds to a patent document – characterized by its title, abstract; and edges represent meaningful relationships between patents. This dual-modality data representation is crucial for patent analysis: the textual attributes X capture the detailed semantic information of inventions, while the graph structure E encodes how patents are interrelated, such as which innovations build upon others or how much similarities are in the title or abstract. A TAG-based approach to patent classification aims to leverage both sources of information, learning classifiers that understand a patent’s content in context of the network of prior art. However, TAG-based patent classification poses unique challenges not present in traditional single-modality classification. First, there is the challenge of dual modality integration: models must jointly handle natural language data and graph-structured data. Patent text is lengthy and complex, containing domain-specific terminology. Designing algorithms that can effectively fuse these modalities—so that relevant textual features inform graph learning and vice versa—is non-trivial. Naively combining a language model with a GNN can lead to excessively large models or training procedures that are difficult to optimize. Indeed, recent studies have noted that coupling large language models with graph neural networks poses computational challenges, often requiring many parameters and careful coordination between text encoding and graph propagation. The second major challenge is scale and computational efficiency. Real-world patent graphs are enormous: the United States Patent and Trademark Office (USPTO) database ¹, for example, contains millions of patents. Learning over

¹<https://www.uspto.gov/patents/search/classification-standards-and-development>

such a graph with high-dimensional text embeddings for each node demands very high memory and processing power. Running a GNN on a million-node graph is itself resource-intensive, and if each node’s features are derived from a transformer-based language model, the cost is multiplied. Training and inference in this setting can incur high computational cost and energy consumption, making it infeasible to naively apply off-the-shelf deep models. For instance, using a modern LLM to encode every patent’s text would require billions of model queries or an expensive distributed inference setup, which remains intractable for many large-scale datasets. Similarly, full-batch graph training on the entire graph network is impractical due to memory limits, necessitating better efficient techniques.

The need for efficient and scalable methods is therefore at the forefront of TAG-based patent classification research. The key problem is how to preserve the rich semantic information of patent texts and the structural context from the patent graph, without exploding the model complexity. In other words, the question we are answering in this report is *how can we build a model that performs accurate multi-modal integration (text + graph) for classification in cost effective setting*. This work addresses that gap by focusing on energy-efficient, scalable algorithms for patent classification on text-attributed graphs. The problem can be stated as developing a patent classification model that effectively integrates textual and graph-based information to achieve high accuracy, while employing computational and energy-efficient techniques to ensure scalability. In tackling this problem, we aim to enable practical, large-scale patent classification that harnesses the best of natural language understanding and graph analytics, thereby advancing the state of the art in intelligent patent information management.

3 Contributions

The contributions of our work is as follows-

- Provided an extensive review of current works in TAG classification, along with an overview of how Patent Analysis is directly relevant to solve real world problem. ✓
- Currently there is no text-attributed graph (TAG) for Patent Analysis. We have built a high-quality TAG dataset for patent focused on downstream tasks like classification and retrieval. ✓
- We provide the dataset and code at this [github repo](#). ✓
- We provide two distinct pathways for energy-efficiency- one by treating the language-model and graph-model components as separate modules and applies well-understood distillation and pruning techniques to each. The second leverages the state-of-the-art TAPE framework [6], utilizing the LLM-to-LM interpreter paradigm to deliver a more integrated and energy-efficient solution. ✓

4 Dataset Description

4.1 Patent Classification

The Cooperative Patent Classification (CPC) system organizes patents into a hierarchical structure, with the highest level being sections labeled by capital letters ranging from A to Y. There are a total of nine main CPC sections: A (Human Necessities), B (Performing Operations; Transporting), C (Chemistry; Metallurgy), D (Textiles; Paper), E (Fixed Constructions), F (Mechanical Engineering; Lighting; Heating; Weapons), G (Physics), H (Electricity), and Y (General Tagging of New Technological Developments). Each section is further divided into classes, subclasses, groups, and subgroups, enabling increasingly fine-grained categorization of technological domains. Table 1 shows an example of the hierarchical categorization. Instead of concentrating on broad CPC sections, our focus is on tackling the more technically challenging task of subclass-level classification.

4.2 Data Collection

We construct our dataset by processing patent data from the PatentsView platform, focusing on patents granted in the year 2022. Specifically, we extract and merge information from two core files: `g_patent` and `g_cpc_current`. From `g_patent`, we collect metadata such as the patent ID, title, abstract and the `g_cpc_current` file is used to extract CPC classification information. We chose one of the

Table 1: An example of how Section A is organized in the Cooperative Patent Classification (CPC) system.

Level	Code	Category
Section	A	Human Necessities
Class	A61	Medical or Veterinary Science: Hygiene
Sub-class	A61C	Dentistry; Apparatus or Methods for dental hygiene
Group	A61C1	Dental machines for boring or cutting
Sub-group	A61C 1/0053	Portable units

highest granted patent class A61. We then filter and select patents belonging to CPC classes with at least 1000 examples to ensure a balanced representation for downstream learning tasks. Our final dataset includes patents from CPC class A61 and its subclasses A61B, A61F, A61K, and A61L, along with other high-frequency classes such as M, N, P and Q. The description for the subclasses are provided in Appendix A. Each entry in the dataset contains four fields: patent ID, title, abstract, and CPC class. After filtering, the dataset comprises approximately 8,000 patent records which makes it suitable for patent classification task.

To construct the graph, we begin by using the pre-trained sentence transformer models to generate semantic embeddings. For each patent, we merge the title and abstract into a single textual input, which is then passed through the model to obtain a dense embedding representation. These embeddings serve as the node features in our graph. We create edges between nodes based on cosine similarity: if the cosine similarity between two nodes exceeds a threshold of 0.7, an undirected edge is added between them. The final graph consists of 8300 nodes (each representing a patent) and 6,035 edges, capturing semantic relationships between patent documents based on their textual content.

4.3 Dataset Representation

We formally construct the patent text-attributed graph (TAG) as a directed semantic similarity graph, where each patent is represented by a node, and edges encode semantic similarity relationships derived from textual content. Specifically, each node includes textual attributes comprising the concatenation of its title and abstract. We define the semantic patent TAG formally as:

$$\mathcal{G}^{\text{patent}} = (\mathcal{V}^p, A, \{s_i\}_{i \in \mathcal{V}^p}), \quad (1)$$

where $\mathcal{V}^p$ is the set of patent nodes, each node $i \in \mathcal{V}^p$ corresponding to a unique patent document. The adjacency matrix $A \in \mathbb{R}^{|\mathcal{V}^p| \times |\mathcal{V}^p|}$ encodes semantic relationships between patent nodes, where an entry A_{ij} indicates the existence of a semantic similarity link between patents i and j . Formally, this semantic similarity-based adjacency is defined as:

$$A_{ij} = 1 \quad \text{if} \quad \text{sim}(s_i, s_j) > \tau \quad \text{and} \quad i \neq j, \quad \text{else} \quad 0 \quad (2)$$

Here, τ denotes a predefined threshold, and $\text{sim}(\cdot, \cdot)$ is a semantic similarity function (e.g., cosine similarity computed using embeddings derived from pre-trained language models). Each node $i \in \mathcal{V}^p$ is associated with a textual sequence $s_i \in \mathcal{D}^{L_i}$, constructed by concatenating its title and abstract texts. Formally, this sequence is given by $s_i = \mathcal{T}(i) \parallel \mathcal{A}(i)$, where $\mathcal{T}(i)$ and $\mathcal{A}(i)$ represent the title and abstract of patent node i , respectively; $\parallel$ denotes the concatenation operator; and L_i indicates the total length of the combined textual sequence. The dictionary $\mathcal{D}$ denotes the vocabulary set derived from all textual content in the dataset. This TAG representation explicitly integrates textual semantic content and structural connectivity into a unified framework, enabling effective downstream tasks including patent node classification, retrieval, and representation learning.

5 Methodology

When working with Text-Attributed Graphs (TAGs), it is crucial to recognize that the classification task inherently involves both textual data and graph-structured data. Accordingly, an effective solution must integrate **Language Models (LMs)** with **Graph Neural Networks (GNNs)**. There are basically two type of frameworks-

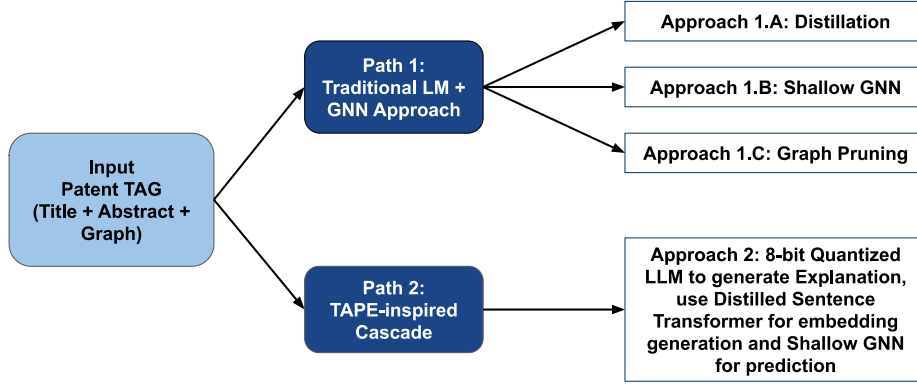


Figure 2: Proposed energy-efficient frameworks for Patent TAG classification

- **Iterative Methods** Train the (L)LM and the GNN together, jointly. LLM features and GNN features evolve together.
- **Cascading Methods** (L)LM is used to extract node features and then used by a GNN for downstream task. LLM features are fixed after extraction.

Iterative methods, which involve joint training of Language Models (LMs) and Graph Neural Networks (GNNs), are often complex and computationally expensive, particularly when operating over large graphs. Moreover, their high memory requirements and intricate optimization dynamics make them poorly scalable to real-world, large-scale settings such as patent citation networks. To address these limitations, we focus on designing energy-efficient Text-Attributed Graph (TAG) classifiers from the perspective of cascading methods, where the LM and GNN are trained in separate stages.

In our methodology shown in Figure 3, we explore two complementary paths toward building an energy-efficient pipeline. The first path takes a modular approach—treating the language model and graph model as separate components—and applies well-known efficiency techniques to each. We compress large language models into smaller, distilled transformers, quantize them for faster inference, prune the graph to retain only the most relevant connections, and rely on shallow GNNs to minimize overhead. The second path takes a more holistic view, inspired by TAPE’s [6] LLM-to-LM interpreter design. Here, we rework each part of the pipeline with energy efficiency in mind: using a quantized LLM to generate explanations, a lightweight sentence encoder to interpret them, and a shallow GNN to complete the reasoning. Together, these two approaches reflect both incremental improvements and architectural simplification—all aimed at making large-scale graph-based reasoning more practical and sustainable.

5.1 Approach 1.A: LM Distillation

In order to minimize the computational overhead during node feature extraction, we replace large-scale LMs such as *SBERT-large* and *DeBERTa* with smaller, distilled alternatives, namely models like *MiniLM*, *DistilBERT*, and *SBERT-L12* or *L-6*. These models are specifically designed to retain most of the representational power of their larger counterparts while significantly reducing the number of parameters, layers, and hidden dimensions. To specify, by adopting DistilBERT, or miniLM for node feature extraction, we significantly reduce the number of model parameters and the overall FLOPs, which in turn lowers energy consumption during training and inference, while still maintaining competitive performance on downstream graph-based tasks.

- **BERT** [2] is pre-trained on large corpora using two unsupervised tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). In MLM, 15% of the tokens in the input are masked, and the model learns to predict these masked tokens. NSP involves predicting whether two sentences appear consecutively in the original text. This pre-training

enables BERT to capture deep bidirectional representations, making it effective for various downstream tasks like question answering and sentiment analysis. However, BERT’s substantial size and complexity lead to high computational costs and energy consumption, making it less suitable for deployment in resource-constrained environments.

- **DistilBERT** [11] is a distilled version of BERT-base and comprises 6 Transformer encoder layers compared to the 12 layers in BERT-base. It maintains a hidden size of 768 dimensions and contains approximately 66M parameters. Despite its reduced size, DistilBERT preserves about 97% of the downstream task performance seen with BERT while achieving a 60% faster inference speed and 40% smaller model size. The knowledge distillation process involves training the smaller DistilBERT model to mimic the behavior of the larger BERT model. This is done by minimizing the difference between the student (DistilBERT) and teacher (BERT) models’ output distributions. As a result, DistilBERT achieves faster inference times and lower energy consumption
- **SBERT-L12** [12] is a variant of Sentence-BERT, which modifies the BERT architecture to generate semantically meaningful sentence embeddings. While BERT is designed for token-level tasks, SBERT introduces a pooling operation over the token embeddings (typically mean pooling) to produce fixed-size sentence embeddings. SBERT-L12 utilizes the BERT-base model with 12 layers, maintaining the same architecture and parameter count (approximately 110 million parameters). To optimize for sentence similarity tasks, SBERT is fine-tuned using Siamese and triplet network structures, enabling it to learn semantically meaningful embeddings that can be compared using cosine similarity. While SBERT-L12 offers high performance in semantic textual similarity tasks, its computational complexity and energy consumption are comparable to BERT-base, which may be a consideration for deployment in energy constrained environments.
- **SBERT-L6** is a more compact version of Sentence-BERT, designed to provide a balance between performance and efficiency. It employs a 6-layer transformer model, such as MiniLM-L6 [12], resulting in a significantly reduced parameter count. Despite its smaller size, SBERT-L6 maintains competitive performance in sentence similarity tasks. A common configuration, MiniLM-L6, consists of 6 Transformer layers and a hidden size of 384 or 768 dimensions, leading to a model with approximately 22M–33M parameters. By operating at a smaller hidden dimension d' , MiniLM reduces the per-layer self-attention computational complexity from $O(T^2d)$ to $O(T^2d')$, where T is the input sequence length. The smaller model size leads to faster inference times and lower memory consumption, making it suitable for deployment in resource-constrained environments.

5.2 Approach 1.B: Shallow Graph Neural Networks

Traditional Graph Neural Networks (GNNs) commonly utilize multiple layers (often 5–6), each recursively aggregating information from neighboring nodes. While deeper architectures can effectively capture complex graph patterns, they significantly increase computational costs due to their per-epoch complexity of $O(L|E|d)$, with L layers, $|E|$ edges, and d feature dimensions. To enhance efficiency, we propose adopting shallow GNN architectures—specifically, two-layer models including Graph Convolutional Networks (GCN), GraphSAGE, and Graph Isomorphism Networks (GIN). These shallow architectures efficiently capture information from immediate neighbors and second-order neighbors, substantially reducing computational complexity while maintaining predictive power. We also incorporate an early stopping mechanism during training. This method involves monitoring the model’s performance on a validation dataset and stopping training once no performance improvement is observed over a specified number of epochs (patience threshold). Early stopping prevents unnecessary computational expenditures, reduces the risk of overfitting, and ensures efficient utilization of resources.

5.3 Approach 1.C: Graph Pruning

Further computational efficiency gains are possible by addressing the dense nature of patent graphs derived from citations or textual similarities. These dense graphs typically involve nodes with numerous connections, substantially increasing the computational load given the linear complexity $O(|E|)$ per graph convolution layer. To mitigate these costs, we apply an *L1 unstructured pruning* technique. This approach systematically removes weights with the lowest magnitudes from a pre-

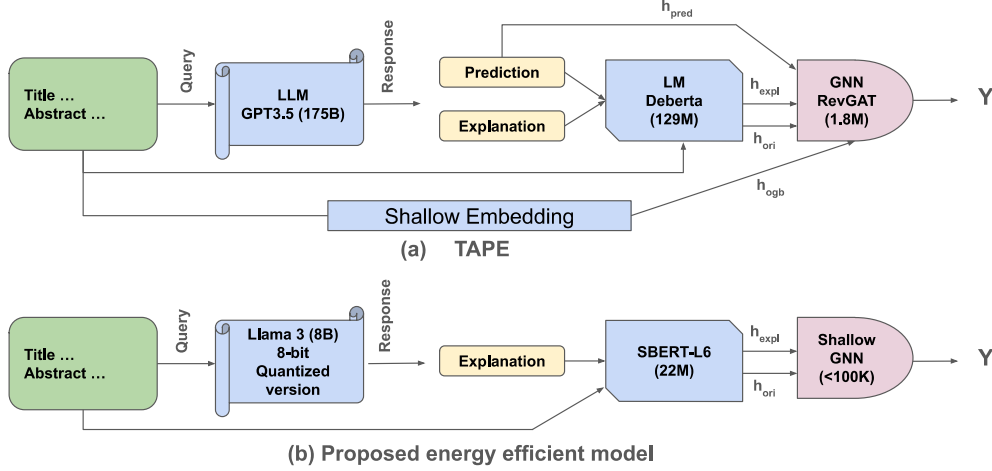


Figure 3: Overview of the TAPE model which is the current SOTA for TAG classification. It leverages GPT3.5 to enhance representation learning on Text-Attributed Graphs (TAGs). Our proposed model builds upon the TAPE framework by integrating insights from Approaches 1.A, 1.B, and 1.C and removing redundant parts, with a particular focus on energy-efficient learning.

trained GNN model. By varying sparsity levels from 0% (no pruning) up to 90%, we significantly reduce model complexity. After pruning, we fine-tune the remaining weights to restore any lost performance and ensure robustness. This pruning strategy effectively reduces the total number of parameters, thus lowering computational and memory requirements. Additionally, pruning acts as implicit regularization, potentially enhancing the generalization capability of the model by eliminating redundant or less critical parameters. The theoretical and empirical basis for pruning, as explored in existing literature such as the Graph Lottery Ticket Hypothesis, indicates that considerable parameter reductions can be achieved with minimal degradation in predictive accuracy.

5.4 Approach 2: Energy Efficient Modification to TAPE

Now we focus on a LLM based text embedding approaches like TAPE, where each node’s textual attributes, such as titles and abstracts, are input into an LLM (e.g., GPT-3.5) through carefully designed prompts. The LLM performs zero-shot classification and provides textual explanations for its predictions. These explanations, rich in semantic and contextual information, are then processed by a smaller, fine-tuned language model (LM), like DeBERTa, which acts as an interpreter. This interpreter transforms the explanations into dense vector representations that encapsulate the reasoning of the LLM. These enriched node features are subsequently utilized by downstream graph neural networks (GNNs) for tasks such as node classification. By decoupling the heavy computational load of LLMs from the training pipeline and focusing on extracting and translating their explanatory insights, we can achieve state-of-the-art performance.

As our focus is more on making TAGs energy efficient and easily scalable to patent classification tasks, we provide a more energy-efficient version of TAPE based on our analysis in Approach 1. Our proposed model introduces comprehensive energy-efficient improvements to the TAPE framework while maintaining its core representational learning capabilities. The original TAPE architecture, as shown in Figure 3(a), relies on computationally intensive components that limit its deployment in resource-constrained environments. Our redesign, illustrated in Figure 3(b), systematically addresses these limitations through a series of targeted modifications aimed at reducing the model’s energy footprint without compromising its effectiveness on TAG-based tasks.

At the input processing stage, we replace the massive GPT3.5 (175B parameters) with an 8-bit quantized version of open-sourced Llama 3 (8B parameters). This substitution achieves a remarkable 95% reduction in model size while the implementation of 8-bit quantization further enhances efficiency by reducing the memory bandwidth requirements by 75% compared to standard 32-bit floating-point operations. Quantization also accelerates matrix multiplications—the dominant operation in transformer

Table 2: Performance comparison of GNN models on patent classification using different LM embeddings (BERT, DistilBERT, SBERT-L12, SBERT-L6) and deep GNN (6 layers).

Model	BERT		DistilBERT		SBERT-L12		SBERT-L6	
	Acc	Time	Acc	Time	Acc	Time	Acc	Time
LM-only (MLP)	0.54	53.86	0.53	31.27	0.52	29.06	0.47	17.82
GCN	0.53	56.86	0.55	33.91	0.52	30.94	0.57	19.59
GAT	0.15	93.08	0.15	69.60	0.15	63.79	0.15	50.77
GraphSAGE	0.52	58.49	0.53	34.77	0.53	33.31	0.53	21.37
GIN	0.24	59.75	0.25	33.77	0.35	32.08	0.41	20.47

architectures—by enabling more efficient fixed-point arithmetic operations that require substantially less energy than their floating-point counterparts. This parameter reduction, when combined with Llama 3’s refined architecture, should preserve the semantic richness at a level needed for effective text representation in graph contexts.

We further streamline the architecture by eliminating the prediction branch entirely, focusing exclusively on explanation generation. The process of explanation generation has been shown in Appendix C. This architectural simplification not only reduces computational overhead but also creates a more direct information flow that minimizes redundant processing pathways. Our reasoning is that high-quality explanations should contain sufficient information to derive predictions when needed, making the separate prediction branch computationally redundant. This insight enabled us to reduce both inference time and energy consumption without sacrificing model utility.

The subsequent sentence transformer undergoes substantial downsizing, transitioning from DeBERTa (129M parameters) to the more compact SBERT-L6 (22M parameters). This 83% parameter reduction is achieved through architectural optimizations that preserve the model’s ability to process explanation embeddings effectively. The choice of SBERT-L6 is justified by the results in Table 3 from Approach 2.B. SBERT-L6 employs distillation techniques that capture the essential representational power of larger models while eliminating redundant parameters. We further enhance efficiency through sparse attention mechanisms that reduce the quadratic complexity typically associated with transformer architectures to a more manageable near-linear relationship, significantly reducing both memory requirements and computational load during inference.

The final efficiency improvement occurs in the graph neural network component, where we replace the complex RevGAT architecture (1.8M parameters) with our ultra-lightweight Shallow GNNs (<100K parameters). This 94% parameter reduction is accomplished through careful analysis of graph feature propagation requirements for TAG-based tasks. Our Shallow GNN implements a simplified message-passing framework that limits propagation to essential neighborhood features, replacing the multi-hop information aggregation of RevGAT with a more focused single-hop approach enhanced by learnable scaling factors. This modification substantially reduces the computational complexity from $O(V^2E)$ to $O(V + E)$, where V represents nodes and E represents edges, enabling significantly faster processing of large graphs with minimal energy expenditure.

We further optimize the overall architecture by eliminating the separate shallow embedding block present in the original TAPE model. Instead, our approach integrates embedding functionality directly into the initial text processing stage, reducing data transfer requirements between components and minimizing memory access operations—a significant source of energy consumption in deep learning pipelines. This integration also reduces the activation memory footprint during inference, allowing the model to process larger graphs without exceeding memory constraints on resource-limited devices.

6 Numerical Results

6.1 Analysis for 1.A: LM Distillation

First, We present a comparative analysis of different GNN models utilizing various LM embeddings from BERT to SBERT-L6. The focus is on identifying configurations that are not only accurate but also computationally efficient. Among all evaluated setups, the combination of GCN with SBERT-L6 stands out as the most energy-efficient high-performance model. It achieves the highest accuracy

Table 3: Performance comparison of GNN models on patent classification using different LM embeddings (BERT, DistilBERT, SBERT-L12, SBERT-L6) and shallow GNN (2 layers).

Model	BERT		DistilBERT		SBERT-L12		SBERT-L6	
	Acc	Time	Acc	Time	Acc	Time	Acc	Time
LM-only (MLP)	0.54	53.86	0.53	31.27	0.52	29.06	0.47	17.82
GCN	0.53	56.01	0.54	30.45	0.51	29.01	0.55	16.58
GAT	0.49	63.67	0.53	37.60	0.54	34.49	0.56	22.69
GraphSAGE	0.55	57.79	0.54	35.09	0.54	30.74	0.58	18.05
GIN	0.46	58.01	0.46	33.50	0.56	31.97	0.57	18.63

(0.57) across all methods while maintaining a low time of 19.59s, making it an ideal candidate for scenarios that require both speed and precision. In contrast, GAT, while generally considered a powerful architecture, fails to deliver competitive accuracy across all LM backbones (consistently around 0.15) and exhibits the highest times (up to 93.08s with BERT), making it the least energy-efficient choice. The result supports the use of SBERT-L6, as a compact energy-efficient alternative of computationally expensive BERT.

6.2 Analysis for 1.B: Shallow GNN

Compared to the deep GNN results in Table 2, Table 3 highlights a compelling advantage of using shallow architectures in terms of both computational efficiency and competitive performance. Across all language model (LM) backbones, shallow GNNs (specifically 2-layer GCN, GraphSAGE, and GIN) achieve inference times that are significantly lower than their 6-layer counterparts—often reducing latency by nearly half. For instance, GCN with BERT drops from 56.86 s (deep) to 56.01 s (shallow), and with SBERT-L6, the time drops even further—from 19.59 s to 16.58 s. These savings become crucial in large-scale or resource-constrained environments. Importantly, this efficiency gain does not come at the cost of accuracy. In fact, shallow GNNs match or even surpass their deeper equivalents when paired with distilled or compact language models. For example, GraphSAGE combined with SBERT-L6 achieves the highest accuracy across all configurations at 0.58, outperforming its deeper variant and even deeper GCNs. GIN and GCN also show moderate accuracy gains when switching from deep to shallow structures under SBERT-L6, all while benefiting from faster inference. These observations validate our methodological decision to prefer shallow GNN architectures: they deliver competitive or superior accuracy while dramatically reducing the energy and time costs associated with graph-based learning.

6.3 Analysis for 1.C: Graph Pruning

Table 4 illustrate the effect of L1 unstructured pruning on GCN models using BERT and SBERT-L6 embeddings, respectively. In both cases, pruning leads to significant reductions in parameter count and inference time, highlighting its utility for energy-efficient learning. For GCN with BERT (Table 4), accuracy remains stable at 0.54 up to 50% pruning, even as the parameter count is halved from 49,664 to 24,832, and inference time drops from 20.16 to approximately 15. In the case of SBERT-L6 (Table 6), pruning yields even more favorable results: the model maintains or improves its accuracy (up to 0.56) at 30–70% sparsity, while inference time is reduced to under 9 ms. However, at 90% pruning, accuracy drops noticeably, suggesting a threshold beyond which aggressive pruning degrades performance. Some inconsistency in inference time—especially in Table 4—may be attributed to the relatively small dataset size and suboptimal sparse matrix handling in the underlying PyTorch execution, which can mask the expected computational savings from pruning. Additional results on GAT model can be found on Appendix B.1.

6.4 Analysis for Approach 2: Energy Efficient Modification to TAPE

For this implementation, one of the main challenges was the limitation imposed by GPU availability and API access. As a result, instead of using a quantized version of GPT, we opted for the openly available 8-bit quantized version of LLaMA 3. While we initially planned to conduct a detailed comparison with the standard 32-bit floating-point version, even the 8-bit model required over 60

Table 4: Comparison of GCN Performance Under L1 Pruning and Fine-Tuning Using BERT vs. SBERT-L6 Embeddings

Prune (%)	BERT				SBERT-L6			
	Acc	#Params	Sparsity (%)	Time	Acc	#Params	Sparsity (%)	Time
0	0.54	49,664	0.00	20.16	0.54	25,088	0.00	17.24
30	0.54	34,764	30.00	15.09	0.56	17,561	30.00	13.06
50	0.54	24,832	50.00	15.35	0.55	12,544	50.00	8.89
70	0.53	14,900	70.00	18.96	0.56	7,527	70.00	8.76
90	0.50	4,966	90.00	14.47	0.48	2,509	90.00	8.86

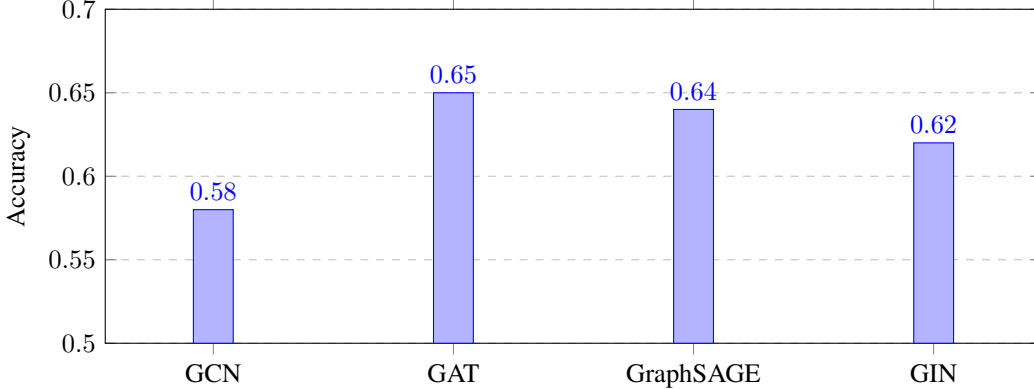


Figure 4: Accuracy of Different GNN Architectures using the energy-efficient proposed model (Approach 2)

hours to complete the explanation generation, given our setup. The full model of Llama-3 contains 70B parameters, which is not possible to be run on Colab’s T4 GPU due to memory constraints. We processed 500 prompts at a time using a Colab T4 GPU via Hugging Face’s API due to Colab’s time constraints. The results of our proposed approach are shown in Figure 4, where it outperforms the methods outlined in Approach 1. However, it’s important to note that the classification tasks evaluated by TAPE (eg. arxiv topics) were relatively easier for LLMs to interpret, unlike patent classification. The patent domain poses a unique challenge due to fine-grained distinctions between subclasses under the same top-level class, often leading to semantic overlap. A representative example of this difficulty is included in Appendix D.

7 Conclusion & Future Works

In this report, we proposed an energy-efficient framework for patent classification on Text-Attributed Graphs (TAGs), combining shallow GNNs, pruning techniques, and a redesigned TAPE architecture. We introduced the first high-quality, open-access TAG dataset tailored for downstream patent tasks like classification and retrieval. Our results show that shallow models such as 2-layer GCN and GraphSAGE, when paired with compact LMs like SBERT-L6, achieve strong accuracy at much lower inference cost. L1 unstructured pruning further reduces parameters and latency without sacrificing performance, making the approach scalable for low-resource environments. We also reengineered the TAPE model with energy-aware updates: quantized LLMs, a distilled explanation encoder, removal of redundant branches, and a shallow GNN replacing RevGAT. These changes led to over 90% model size reduction while maintaining strong representational performance. Lastly, we found that explanation-only reasoning is sufficient for graph-based classification, enabling a more streamlined and efficient pipeline. As future work, we plan to explore structured pruning methods aligned with hardware-aware constraints to further enhance speedups. Additionally, we aim to investigate retrieval-augmented prompting and dynamic graph construction to better adapt to evolving patent corpora and multimodal data.

References

- [1] Hamid Bekamiri, Daniel S Hain, and Roman Jurowetcki. Patentsberta: A deep nlp based hybrid model for patent distance and classification using augmented sbert. *Technological Forecasting and Social Change*, 206:123536, 2024.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [3] Lintao Fang, Le Zhang, Han Wu, Tong Xu, Ding Zhou, and Enhong Chen. Patent2vec: Multi-view representation learning on patent-graphs for patent classification. *World Wide Web*, 24(5):1791–1812, 2021.
- [4] Annemarie Friedrich and Subhash Chandra Pujari. Device and method for determining a knowledge graph. US Patent App. 17/400,631, 2022. Publication number US20220092440A1.
- [5] William L Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] Xiaoxin He, Xavier Bresson, Thomas Laurent, Adam Perold, Yann LeCun, and Bryan Hooi. Harnessing explanations: LLM-to-LM interpreter for enhanced text-attributed graph representation learning. In *The Twelfth International Conference on Learning Representations*, 2024.
- [7] Eleni Kamateri, Michail Salampasis, and Konstantinos Diamantaras. An ensemble framework for patent classification. *WPI*, 75:102233, 2023.
- [8] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [9] Jieh-Sheng Lee and Jieh Hsiang. Patentbert: Patent classification with fine-tuning a pre-trained bert model. *arXiv preprint arXiv:1906.02124*, 2019.
- [10] Shaobo Li, Jie Hu, Yuxin Cui, and Jianjun Hu. Deeppatent: patent classification with convolutional neural networks and word embedding. *Scientometrics*, 117(2):721–744, 2018.
- [11] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [12] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in neural information processing systems*, 33:5776–5788, 2020.
- [13] Jianan Zhao, Meng Qu, Chaozhuo Li, Hao Yan, Qian Liu, Rui Li, Xing Xie, and Jian Tang. Learning on large-scale text-attributed graphs via variational inference. In *The Eleventh International Conference on Learning Representations*, 2023.

A Subclass Categories for classification

Table 5: Patent Classification Categories Used in This Study

Class	Description
A61B	Diagnosis; Surgery; Identification
A61F	Filters Implantable into Blood Vessels
A61K	Preparations for Medical, Dental or Toiletry Purposes
A61L	Methods or Apparatus for Sterilising Materials or Objects in General
A61M	Devices for Introducing Media into, or onto, the Body
A61N	Electrotherapy; Magnetotherapy; Radiation Therapy; Ultrasound Therapy
A61P	Specific Therapeutic Activity of Chemical Compounds or Medicinal Preparations
A61Q	Specific Use of Cosmetics or Similar Toiletry Preparations

B Additional Results & Explanations

B.1 GNN Pruning

Pruning result on GAT provided in Table. 6 and Table. 7

Table 6: Pruning Results Summary on GAT Model(BERT)

Prune (%)	Accuracy	# Params	Sparsity (%)	Time
0	0.4681	49,664	0.00	14.28
30	0.4952	34,764	30.00	21.16
50	0.5096	24,832	50.00	14.96
70	0.4837	14,900	70.00	21.39
90	0.4934	4,966	90.00	14.90

Table 7: Effect of Pruning and Fine-tuning on GAT Performance(SBERT-L6)

Prune (%)	Accuracy	# Params	Sparsity (%)	Time
0	0.56	25,088	0.00	12.62
30	0.56	17,561	30.00	13.33
50	0.57	12,544	50.00	9.51
70	0.56	7,527	70.00	9.91
90	0.55	2,509	90.00	9.08

C Llama3 Query

Patent Classification Prompt

You are an expert on the US Patent system and have in-depth knowledge of the distinctions between different patent classes. Your goal is to complete the following task:

Task: Given a patent abstract, generate a concise explanation and classify the patent into one of the following predefined classes:

A61B: DIAGNOSIS; SURGERY; IDENTIFICATION

A61F: FILTERS IMPLANTABLE INTO BLOOD VESSELS

A61K: PREPARATIONS FOR MEDICAL, DENTAL OR TOILETRY PURPOSES

A61L: METHODS OR APPARATUS FOR STERILISING MATERIALS OR OBJECTS IN GENERAL

A61M: DEVICES FOR INTRODUCING MEDIA INTO, OR ONTO, THE BODY

A61N: ELECTROTHERAPY; MAGNETOTHERAPY; RADIATION THERAPY; ULTRASOUND THERAPY

A61P: SPECIFIC THERAPEUTIC ACTIVITY OF CHEMICAL COMPOUNDS OR MEDICINAL PREPARATIONS

A61Q: SPECIFIC USE OF COSMETICS OR SIMILAR TOILETRY PREPARATIONS

Patent Abstract:

Patent Abstract

Respond in the following format:

Explanation: <brief reasoning under 100 words >

Example Response:

Explanation: The abstract describes vaccine compositions for treating and/or preventing infections, which is a medical preparation for a specific purpose. The composition comprises bacteria of the Chlamydiaceae family, which have been previously treated by at least one peptidoglycan inhibitor, or extracts of said treated bacteria. This falls under the category of “preparations for medical, dental or toiletry purposes” as described in A61K.

D Subclass Overlap Challenges in Patent Classification

Classifying patents into sub-classes is inherently harder than standard tag classification because the meanings of sub-classes often overlap. For instance, A61K (‘Preparations for medical, dental or toiletry purposes’) and A61Q (‘Specific use of cosmetics or similar toiletry preparations’) can both apply to products with therapeutic or cosmetic effects. In our dataset, one patent describes making a bath bomb with cannabinoids (THC/CBD) that undergo an acid etching process before being mixed with ingredients like sodium bicarbonate. Our model predicted A61K. This wasn’t illogical, as the description highlighted chemical steps, formulation details, and active compounds (THC/CBD), much like patents for medicinal preparations (A61K). Nevertheless, the correct class is A61Q. This class is better suited because the product is intended for topical cosmetic use in a bath. A61Q emphasizes the application context (like skincare or bathing) rather than just the drug-like nature or preparation method. While A61K covers making therapeutic agents, A61Q focuses on their use, especially in non-medical cosmetic contexts like bathing.